

USE CASES V2 — Catalyst (Stakeholders + Flows + Guide)

Version: 2.0 | Status: Ready for Build | Sources: docs/v2/PRD.md V1.1, docs/v2/TECH_SPEC.md V2.0

Scope: Boolean flags only, dev / staging / prod, JS + Python SDKs, light audit. Fetch-on-load only — no SSE, no polling, no push. Out of scope per PRD v2 §8: RBAC, SSO, experiments, scheduling, real-time propagation.

Naming note: this is the *Use Case Specification* (sometimes called *User Flows / Stakeholder Playbook*). PRD says WHAT, TECH_SPEC says HOW, this doc says WHO does WHAT, step by step, with a concrete example you can run in the demo.

Propagation model (read once): the SDK fetches the full flag snapshot during init (on app load) and evaluates from memory. A toggle in the dashboard takes effect on the next load/refresh of the consuming app. No polling, no SSE, no <5s promise — the happy demo is flip -> refresh -> flip.

Source	docs/v2/PRD.md V1.1 + docs/v2/TECH_SPEC.md V2.0
Scope	Boolean flags only, dev / staging / prod, JS + Python SDKs, light audit. Fetch-on-load only.
Propagation	No SSE, no polling. SDK fetches on init; toggle reflected on next load/refresh.

This PDF was exported from v2/USE_CASES.md. The Markdown file remains the source of truth; this PDF is the shareable snapshot for review.

0. Stakeholders + Running Example

Stakeholder	Role in V1	Needs
Release Manager (RM)	No-code operator, owns rollouts	Toggle without deploy, slider, rules, playground
Developer (Dev)	Integrates SDK into apps	init / isEnabled, offline safety
Org Owner (You)	Setup + keys + demo	Signup auto-provision, key rotation, audit
Evaluator (Faculty / Recruiter)	Grades / judges the build	5-minute demo path, proof of sticky rollouts + audit

Running example used everywhere below: flag new-checkout (new checkout page), project default, env prod. Test users: u123 (a@test.com, IN, v2.1), u456 (b@gmail.com, US, v1.9), u789 (c@test.com, IN, v2.0). Eval order (PRD v2 §5): kill OFF -> rules (priority) -> % rollout (murmur(flagKey:userId) % 100) -> default. Reason always returned: disabled | rule:<id> | rollout | default.

UC-1 — Setup: org -> project -> envs -> keys (Owner)

Goal: going from zero to evaluable flags in <2min. **Precondition:** API + Postgres (+ optional Redis) running (docker compose up -d postgres, prisma migrate dev, npm dev).

- 1 POST /api/v1/auth/signup {email, password, orgName:"acme"} -> auto-creates org acme + project default + envs dev/staging/prod + 3 sdk_keys. JWT set as httpOnly cookie.
- 2 Dashboard -> Keys page -> copy cp_prod_xxx (shown once; only prefix + hash stored afterwards).
- 3 Create flag: POST /api/v1/projects/:pid/flags {key:"new-checkout", name:"New checkout", defaultValue:false} -> creates 3 FlagEnvState rows (enabled=true, percentage=0).
- 4 Hand sdk_key (prod) + projectId to Dev for UC-5/UC-6.

Exception: duplicate key -> 409 CONFLICT; bad regex -> 400 VALIDATION. Switching env in UI re-fetches scoped bootstrap — state never leaks across envs (TECH_SPEC v2 §8).

UC-2 — Kill switch: stop a broken release (RM)

Goal: broken checkout in prod -> OFF instantly, no redeploy. **Precondition:** UC-1 done, demo apps running with prod sdk_key.

- 1 Incident: errors spike on new checkout.
- 2 Dashboard -> Flags -> new-checkout -> tab prod -> Kill toggle **OFF** (PUT /flags/:id/state/prod {enabled:false}).
- 3 Backend: state.version++, env.version++, audit flag.toggled {before:{enabled:true}, after:{enabled:false}}.
- 4 Consumers: **on their next load/bootstrap**, isEnabled('new-checkout', ...) returns {value:false, reason:"disabled"}. A reloaded page shows old checkout gone immediately.
- 5 Verify: playground userId=u123 -> false via disabled; audit History tab shows who/when/old->new.

Example check: POST /api/v1/evaluate {flagKey:"new-checkout", context:{userId:"u123"}} -> {value:false, reason:"disabled", version:7}.

UC-3 — Gradual rollout 1% -> 50% -> 100% (RM)

Goal: canary then ramp with sticky bucketing. **Precondition:** kill ON (enabled=true), no rules yet.

- 1 Set prod percentage=1 (slider + numeric input). Non-targeted users hash `murmur("new-checkout:<userId>") % 100 < 1` -> ~1% see true, reason:"rollout". Same user, same result across 1000 evals.
- 2 Watch demo apps as users reload, then ramp 1 -> 50 -> 100. 0% = OFF, 100% = ON for non-targeted users with user Id.

- 3 Gate: distribution +/-2% at 10k synthetic users (tested in Wk2 vectors).

Worked mini-example (illustrative buckets, real hash decides): at 50%, `u123 -> bucket 12 -> true` (rollout), `u456 -> bucket 71 -> false` (default), `u789 -> bucket 44 -> true` (rollout).

Exception: context without `userId` -> rollout skipped -> `defaultValue` (documented simple semantic; no anonymous bucketing in V1).

UC-4 — Targeted beta: internal users first (RM)

Goal: IF email endsWith `@test.com` OR (country==IN AND version>=2.0) THEN true. Model as two rules (OR across rules, AND within a rule — fixed in V1):

- Rule #0 (priority 0): [{email, endsWith, "@test.com"}] -> serve true.
- Rule #1 (priority 1): [{country, equals, "IN"}, {version, gte, "2.0"}] -> serve true.

- 1 Dashboard -> flag detail -> Rules -> Add rule modal (attribute/operator/value/serve) -> drag-reorder priority -> save (PUT .../rules/reorder renormalizes 0..n, bumps versions, audits).
- 2 Playground matrix (POST /flags/:id/test-eval {env:"prod", context}):

user	result	reason
u123 a@test.com IN v2.1	true	rule:#0 (first match wins)
u789 c@test.com IN v2.0	true	rule:#0
u456 b@gmail.com US v1.9	falls to rollout/default	rollout or default

- 3 Missing attribute (e.g. no country) -> that condition is false -> rule safely skips, never throws.
- 4 Limit (documented): version >= 2.0 is naive numeric-then-string compare, NOT semver — 2.10 vs 2.9 can surprise; semver is Phase 2.

UC-5 — Integrate the JS SDK (Dev)

Goal: `isEnabled()` with fetch-on-load and offline safety. **Precondition:** prod `sdkKey` + `projectId` from UC-1.

```
await catalyst.init({ sdkKey: 'cp_prod_xxx', baseUrl: 'http://localhost:3001', env: 'prod', projectId: 'pr
const on = catalyst.isEnabled('new-checkout', { userId: 'u123', email: 'a@test.com', country: 'IN', versio
```

Flow: page load -> init -> GET /bootstrap (If-None-Match: version -> 304 if unchanged) -> cache memory + disk -> local eval <10ms. New toggles are picked up on the next init/page refresh. Network loss -> last cached snapshot -> `defaultValue`. Never throws on eval.

No onUpdate, no polling, no SSE — nothing to wire for live updates; refresh the page.

UC-6 — Integrate the Python SDK (Dev)

Same contract, Python idioms (`requests`, disk cache `~/ .catalyst/cache.json`, no background thread — nothing to poll; never raises on eval):

```
catalyst.init(sdk_key="cp_prod_xxx", base_url="http://localhost:3001", env="prod", project_id="prj_123")
on = catalyst.is_enabled("new-checkout", {"userId": "u123", "email": "a@test.com", "country": "IN", "version": "1.0.0" })
```

Parity gate: both SDKs pass shared `eval.vectors.json` (same murmur seed 0, same operator table as `apps/api/src/lib/eval.ts`).

UC-7 — Debug: playground + audit (Dev / RM)

- 1 User reports `true` unexpectedly -> paste their `userId/email/country/version` into playground -> output `true` via `rule:#1` (names the exact rule).
- 2 Flag detail -> History tab -> timeline `timestamp`, `actor`, `action`, `before->after` (append-only; no edit/delete API).
- 3 Kill precedence reminder: if `enabled=false`, playground shows `disabled` regardless of rules — check kill first.

UC-8 — Rotate a leaked key (Owner / Dev)

- 1 Keys page -> `prod` -> Rotate (confirm) -> old key revoked immediately, new raw key shown once — audit `key.rotated`.
- 2 SDKs get 401 -> `re-init` with new key -> re-bootstrap. Rate limit note: `600/min/key`, read-only (`bootstrap/evaluate` only, no writes via `sdk_key`).

UC-9 — Evaluator demo script, <5 minutes (Faculty / Recruiter)

- 1 (30s) Signup -> auto-provisioned org/project/3 envs + keys. State the theme: deploy once, release gradually.
- 2 (60s) JS demo app (u123) shows old checkout. Toggle prod kill OFF -> **refresh/reload the app** -> it flips to old checkout, `reason:"disabled"`.
- 3 (60s) Kill ON, rollout 10% -> reload two users, different results, same user stable across reloads (sticky). Mention `+/-2% @10k` test.
- 4 (60s) Add beta rule (UC-4) -> playground `a@test.com` -> `true` via `rule:#0`. Show priority reorder.
- 5 (30s) History tab: every click audited `who/when/old->new`. Kill the network -> reload Python demo -> app still runs on cached/default (never crashes).

- 6 Close with scope honesty: boolean-only, naive version compare, no real-time push (refresh to apply), no RBAC — all Phase 2 per PRD v2 §8.

End-to-end flows

Incident — kill a broken release, apply on next load:

- 1 detect the failure (errors spike on new checkout)
- 2 dashboard -> new-checkout -> prod -> kill OFF
- 3 `state.version++` + `env.version++` + audit entry
- 4 consuming apps pick it up on their next load/reload (`reason: "disabled"`)
- 5 verify in playground -> ship fix -> kill ON -> ramp %

First integration — SDK in an afternoon:

- 1 signup -> copy `sdk_key` + `projectId`
- 2 SDK `init` -> bootstrap snapshot cached
- 3 evaluate locally -> `isEnabled` wired to your UI
- 4 flip the kill toggle -> reload the app -> watch it flip

Beta launch — targeted, then ramp:

- 1 create rules -> set priority order
- 2 playground matrix -> confirm `rule:#0` matches
- 3 rollout 10% -> monitor -> 50% -> 100%
- 4 archive the old flag when done (archived flags are excluded from bootstrap/evaluate)

Troubleshooting

Symptom	Cause -> fix
401 on bootstrap	Wrong env key or rotated -> copy current per-env key, check <code>projectId+env</code> pair
App still shows old value after a toggle	By design: fetch-on-load — refresh/reload the app (optionally <code>re-init</code> the SDK)
Rule never matches	Missing context attr -> condition false by design; log the context you actually send
User flaps true/false	<code>userId</code> unstable (session id rotating) -> use stable id; V1 has no anonymous bucketing

Symptom	Cause -> fix
version>=2.0 surprises	Naive compare, not semver -> keep major .minor zero-padded or wait for Phase 2
Offline reload shows stale value	Serving last cached snapshot + defaultValue — intended no-throw behavior